Using Machine Learning for Stock Price Prediction

by

Dev Luhar

MA4K8 Scholarly Report

Submitted to The University of Warwick

Mathematics Institute

January, 2026

Contents

1 Introduction **1**

2 Problem Formulation **2**
2.1 Time Series Problem . . . 2
2.2 Sentiment-Driven Extension . . . 2
2.3 Evaluation Metrics . . . 2

3 Linear Time Series Models **4**
3.1 Autoregressive Models . . . 4
3.2 Moving Average Models . . . 5
3.3 Autoregressive Moving Average Models . . . 5
3.4 Comparison and Limitations of Linear Models . . . 5

4 Neural Networks **6**

5 Training and Optimisation Challenges **8**
5.1 Vanishing and Exploding Gradients . . . 8
5.2 Fitting, underfitting and overfitting . . . 9

6 LSTM **12**
6.1 LSTM Architecture . . . 12
6.2 Data pre-processing . . . 14
6.3 Loss Function Comparison . . . 15
6.4 Layers . . . 16
6.5 Training Procedure and Model Configuration . . . 17
6.6 Analysis of Prediction . . . 19

7 Sentiment-Driven Price Prediction **19**
7.1 Tokenisation . . . 20
7.2 Embeddings and the Transformer Encoder . . . 21
7.3 Input Structure and Data Preparation . . . 24
7.4 Mamba State Space Model . . . 24
7.5 Modelling Assumptions . . . 28

8 Combining Models **28**
8.1 Model Outputs and Notation . . . 28
8.2 XGBoost-Based Forecast Combination . . . 29

9 Analysis of Combined Prediction and Conclusion **29**

10 Conclusion **30**

1 Introduction

Recent years have seen significant progress in artificial intelligence, particularly in deep learning and natural language processing [4, 6]. These developments have transformed how prediction tasks are approached, enabling models to capture complex, non-linear relationships that traditional linear methods struggle to represent. In financial forecasting, such advances provide new tools for improving predictive accuracy.

This report investigates the development of a stock price prediction algorithm that combines time-series modelling with information extracted from financial news. While the broader objective is to design a framework that can be applied to stock price forecasting more generally. Gold is used here as the primary test case and benchmark. For the purposes of evaluation, this study uses the Yahoo Finance ticker GC=F. Although GC=F represents gold futures rather than spot gold itself, it serves as a practical and widely used proxy for gold price movements, making it a suitable benchmark for assessing the effectiveness of the proposed prediction framework.

The core predictive framework is based on historical price data, modelled using Long Short-Term Memory (LSTM) networks [3]. LSTMs are well suited to financial time series as they capture temporal dependencies while mitigating vanishing and exploding gradient issues that affect standard recurrent neural networks.

To improve predictive accuracy, financial news data is incorporated as an additional source of information. News text is first processed using FinBERT, a transformer model specialised for financial sentiment analysis [7], to extract sentiment-aware representations from market-related articles and headlines. These outputs are then passed through a Mamba model [8], which captures sequential structure and longer-range dependencies in the news signal more efficiently.

Rather than acting as an independent forecasting model, this news-based component is designed to complement the LSTM by providing contextual information not present in historical prices alone. In this way, the time-series model captures momentum and longer-term structure, while the FinBERT-Mamba pipeline helps explain sudden deviations caused by external events and shifts in market sentiment.

A Python implementation accompanies this report, providing a complete framework for data preprocessing, feature extraction, model training and evaluation. Model performance is assessed using standard metrics on held-out test data. Particularly focussing on how effectively the proposed stock price prediction algorithm performs when tested on gold as a benchmark asset and whether the inclusion of FinBERT and Mamba-derived news features improves the accuracy of its predictions.

2 Problem Formulation

Let $\{y_t\}_{t\in\mathbb{Z}}$ denote the gold price time series, where $y_t \in \mathbb{R}$ is the closing price at time t. The objective is to model the price evolution and produce forecasts of future prices. We assume the existence of an unknown deterministic function

$$F : \mathcal{I}_{t-1} \to \mathbb{R},$$

such that

$$y_t = F(\mathcal{I}_{t-1}),$$

where $\mathcal{I}_{t-1}$ denotes the information set available immediately prior to time t. This information set may include historical prices, derived features and exogenous variables. This abstract formulation pushes away from the specific choice of inputs, allowing us to select the ideal mapping from available information to the next observed price.

2.1 Time Series Problem

Given a historical dataset $\{y_t\}_{t=1}^{T}$, the objective is to construct an approximation $\hat{F}$ of F that produces forecasts

$$\hat{y}_t = \hat{F}_{TS}(y_{t-1}, y_{t-2}, \ldots, y_{t-n}),$$

where $\hat{y}_t \in \mathbb{R}$ denotes the predicted closing price at time t. The quality of the approximation $\hat{F}$ is assessed using suitable error metrics, introduced in a later section.

2.2 Sentiment-Driven Extension

To augment price-based forecasting with exogenous information, we incorporate textual news data as an additional predictive signal. Let $\mathcal{H}_t$ denote a text-based news headline observed at time t, represented via a numerical embedding. We model the effect of sentiment through a function

$$\hat{F}_{NLP} : \text{Text} \to \mathbb{R},$$

such that

$$\widehat{\Delta y}_t = \hat{F}_{NLP}(\mathcal{H}_t),$$

where $\Delta y_t = y_{t+1} - y_t$ denotes the one-step price gradient. The sentiment-driven output $\widehat{\Delta y}_t$ is treated as an exogenous input to the price forecasting model, allowing textual information to influence predicted price dynamics without replacing the underlying time series structure.

2.3 Evaluation Metrics

Definition 2.1 (Root Mean Squared Error (RMSE))**.** *Let $\{y_i\}_{i=1}^{N}$ be the true observed prices and $\{\hat{y}_i\}_{i=1}^{N}$ the corresponding predicted prices. The Root Mean Squared Error is*

defined as

$$\mathrm{RMSE} = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2}.$$

RMSE measures the average magnitude of price-level errors and penalises larger deviations more heavily [15].

Definition 2.2 (Coefficient of Determination (R^2))**.** *Let $\bar{y} = \frac{1}{N}\sum_{i=1}^{N} y_i$ denote the sample mean of observed prices. The coefficient of determination is*

$$R^2 = 1 - \frac{\sum_{i=1}^{N}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{N}(y_i - \bar{y})^2}.$$

R^2 quantifies the fraction of variance in the observed series explained by the model [16].

Definition 2.3 (Return Mean Squared Error (Return MSE))**.** *Define realised and predicted one-step returns for $i = 2, \ldots, N$ by*

$$r_i = \frac{y_i - y_{i-1}}{y_{i-1}}, \qquad \hat{r}_i = \frac{\hat{y}_i - y_{i-1}}{y_{i-1}}.$$

The return-domain mean squared error is

$$\mathrm{MSE}_{\mathrm{return}} = \frac{1}{N-1}\sum_{i=2}^{N}(r_i - \hat{r}_i)^2.$$

This metric evaluates forecast quality directly in percentage-change space [14].

Definition 2.4 (Directional Accuracy (DA))**.** *Let $\{y_i\}_{i=1}^{N}$ be the true observed values and $\{\hat{y}_i\}_{i=1}^{N}$ be the corresponding predicted values. Directional Accuracy measures the proportion of times the model correctly predicts the direction of change between consecutive observations. It is defined as*

$$\mathrm{DA} = \frac{1}{N-1}\sum_{i=2}^{N}\mathbb{I}(\mathrm{sign}(y_i - y_{i-1}) = \mathrm{sign}(\hat{y}_i - y_{i-1}))\,,$$

where $\mathbb{I}(\cdot)$ denotes the indicator function. Directional Accuracy evaluates the model's ability to correctly anticipate upward or downward movements rather than the magnitude of the prediction error [17].

The objective is to minimise RMSE and Return MSE while maximising R^2 and Directional accuracy.

3 Linear Time Series Models

Linear time series models form a core part of classical forecasting [1]. They assume that the evolution of a time series can be described as a linear function of its past values and random shocks. Despite their simplicity, these models remain widely used due to their interpretability and computational efficiency.

3.1 Autoregressive Models

Definition 3.1 (Autoregressive Process)**.** *An Autoregressive process of order n, denoted AR(n), is defined by*

$$X_t = \sum_{a=1}^{n} \phi_a X_{t-a} + \epsilon_t,$$

where $\{\epsilon_t\}$ is a white noise sequence with $\mathbb{E}[\epsilon_t] = 0$ and $\mathrm{Var}(\epsilon_t) = \sigma^2$ and $\phi_1, \ldots, \phi_n \in \mathbb{R}$ are model parameters.

The AR model assumes that the current value of the process depends linearly on a finite number of its past observations, with randomness captured by the noise term ϵ_t.

Definition 3.2 (Stationarity)**.** *A stochastic process $\{X_t\}_{t\in\mathbb{Z}}$ is called* stationary *if its statistical properties do not change over time.*

Definition 3.3 (Strong Stationarity)**.** *The process $\{X_t\}$ is* strongly stationary *if, for any $k \in \mathbb{N}$ and any time indices $t_1, \ldots, t_k$ and lag n, the joint distributions satisfy*

$$(X_{t_1}, \ldots, X_{t_k}) = (X_{t_1+n}, \ldots, X_{t_k+n}).$$

Definition 3.4 (Weak Stationarity)**.** *The process $\{X_t\}$ is* weakly stationary *if $\mathbb{E}[X_t] = \mu, \mathrm{Var}(X_t) = \sigma^2$, are constant in time and the auto-covariance function $\gamma(k) = \mathrm{Cov}(X_t, X_{t-k})$ depends only on the lag k and not on t.*

Weak stationarity is sufficient for most linear time series models, including AR, MA and ARMA, as it ensures stable second-order structure and enables consistent parameter estimation.

Theorem 3.5 (Stationarity of AR Processes)**.** *An AR(n) process is weakly stationary if and only if the* ***roots*** *of the characteristic polynomial*

$$1 - \phi_1 z - \phi_2 z^2 - \cdots - \phi_n z^n = 0,$$

lie ***outside*** *the unit circle.*

This condition ensures that the contribution of past observations to X_t diminishes as the lag increases, so that shocks do not have a persistent or amplifying effect on the process.

As a result, the variance of the series remains bounded and the process does not exhibit exponential growth or divergence over time.

3.2 Moving Average Models

Definition 3.6 (Moving Average Process)**.** *A Moving Average process of order m, denoted $MA(m)$, is defined by*

$$X_t = \epsilon_t + \sum_{b=1}^{m} \theta_b \epsilon_{t-b},$$

where $\{\epsilon_t\}$ is white noise and $\theta_1, \ldots, \theta_m \in \mathbb{R}$.

Unlike AR models, MA models express the process entirely as a linear combination of current and past shocks. As a result, MA processes are always stationary.

3.3 Autoregressive Moving Average Models

Definition 3.7 (ARMA Process)**.** *An autoregressive moving average process of order (n, m), denoted $ARMA(n, m)$, is defined by*

$$X_t = \sum_{a=1}^{n} \phi_a X_{t-a} + \epsilon_t + \sum_{b=1}^{m} \theta_b \epsilon_{t-b}.$$

ARMA models combine the persistence of AR processes with the shock-driven structure of MA processes, allowing for a richer class of linear dynamics [1].

3.4 Comparison and Limitations of Linear Models

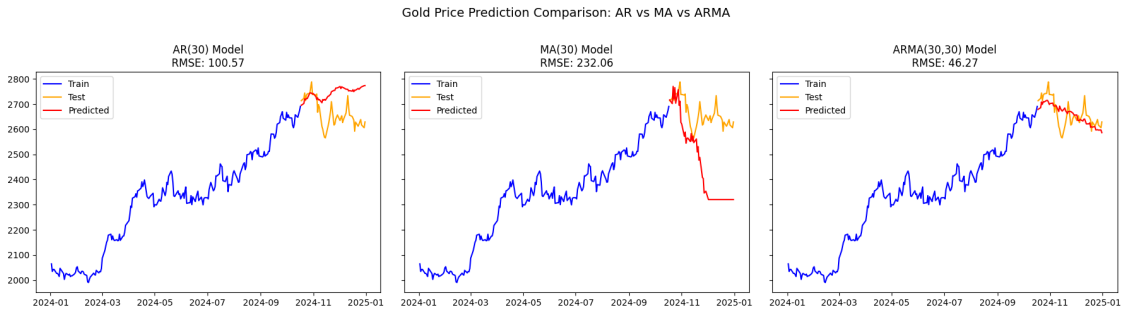

Figure 1: Comparison of AR, MA and ARMA models

Figure 1 compares the behaviour of AR, MA and ARMA models fitted to the same time series. The AR model produces smoother dynamics based on past values, while the MA model responds more sharply to recent shocks. The ARMA model combines both effects and often improves short-term forecasting performance. However, these linear models rely on restrictive assumptions, particularly linearity and time-invariant parameters, which limit their ability to capture the non-linear behaviour, asymmetric shock responses and

structural changes often present in financial time series. As a result, their forecasting performance can weaken in complex and evolving market environments, motivating the use of non-linear and machine learning-based approaches.

4 Neural Networks

Neural networks are flexible function approximators that learn non-linear relationships between inputs and outputs by composing multiple affine transformations with non-linear activation functions.

Definition 4.1 (Neural Network)**.** *A neural network is a parametrised map* $F_\theta : \mathbb{R}^{d_{\text{in}}} \to \mathbb{R}^{d_{\text{out}}}$ *formed by stacking layers of the form*

$$z^{(\ell+1)} = \sigma\Big(W^{(\ell)} z^{(\ell)} + b^{(\ell)}\Big),$$

where $W^{(\ell)}$ *and* $b^{(\ell)}$ *are learnable parameters and* σ *is a non-linear activation function.*

Remark 4.2. *Standard feed-forward networks process each sample independently. This is effective for static data but does not explicitly model temporal dependencies in sequential observations such as daily gold prices.*

To model sequence dynamics in this project, we use recurrent neural networks (RNNs) and in particular LSTMs, to capture long-term dependencies in time-ordered price data.

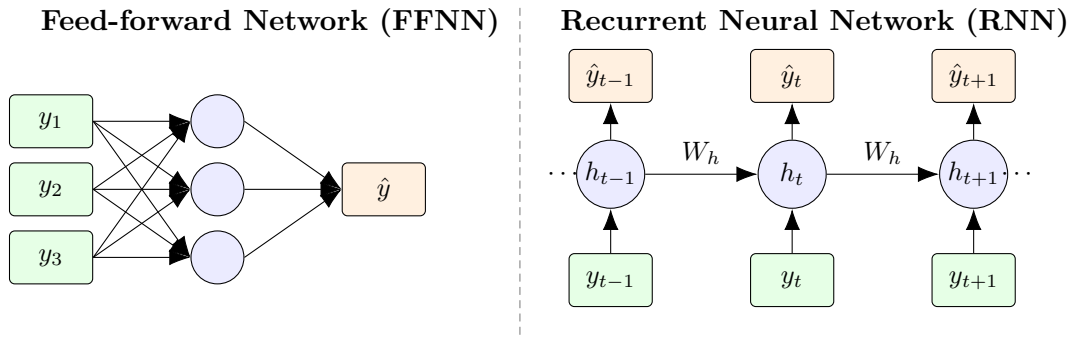

Figure 2: Comparison between a feed-forward neural network (FFNN) and an unrolled recurrent neural network (RNN)

Definition 4.3 (Recurrent Neural Network)**.** *Let* $\hat{F}_\theta : \mathbb{R}^n \to \mathbb{R}$ *denote a recurrent neural network parametrised by* $\theta \in \mathbb{R}^l$*.* n *corresponds to the length of the time window used for prediction, the output is one-dimensional, representing the predicted next closing price.*

Given an input sequence $(y_1, y_2, \ldots, y_T)$*, the network produces a scalar output* $\hat{y}_T = \hat{F}_\theta(y_1, y_2, \ldots, y_T)$*.*

The defining feature of an RNN is the recurrent hidden state with feedback connections that evolves over time, as illustrated in Figure 2.

Lemma 4.4 (Hidden State)**.** *Let $h_t \in \mathbb{R}^{d_h}$ denote the hidden state at time t. The hidden states evolution is given by*

$$h_t = \phi(Uy_t + Wh_{t-1} + b_h),$$

where $U \in \mathbb{R}^{d_h \times 1}$ is the input-to-hidden weight matrix, $W \in \mathbb{R}^{d_h \times d_h}$ is the recurrent weight matrix, $b_h \in \mathbb{R}^{d_h}$ is a bias vector and ϕ is an element-wise activation function such as tanh*, sigmoid or ReLU.*

The bias term provides an additional degree of freedom, allowing the activation function to shift and improving the model's ability to fit the data. While the hidden state captures temporal dependencies, an explicit output is obtained via a linear transformation followed by a non-linearity.

Lemma 4.5 (Output Mapping)**.** *The output at time step t is given by*

$$\hat{y}_t = \phi(Vh_t + b_y),$$

where $V \in \mathbb{R}^{1 \times d_h}$ is the hidden-to-output weight matrix and $b_y \in \mathbb{R}$ is the output bias. In this work, only the final output $\hat{y}_T$ is retained, as the objective is to predict the next closing price.

A crucial structural property of RNNs is that the same parameters are reused at every time step.

Remark 4.6. *The weight matrices U, W and V, along with the bias vectors, are shared across all time steps of the input sequence. The reason for this is that the recurrence relation defining the hidden state is time-invariant, enforcing parameter sharing across time.*

This parameter sharing has important implications for both model complexity and flexibility.

Theorem 4.7 (Variable-Length Sequence Modelling)**.** *Fix the input and hidden dimensions. For each sequence length $T \geq 1$, the unrolled RNN defines a map*

$$\hat{F}_\theta^{(T)} : (\mathbb{R}^{d_{\text{in}}})^T \to \mathbb{R}^{d_{\text{out}}}$$

using the same parameter set $\theta = \{U, W, V, b_h, b_y\}$. Hence, the number of learnable parameters is independent of T.

Proof. For any t, the recurrence and output equations are

$$h_t = \phi(Uy_t + Wh_{t-1} + b_h), \qquad \hat{y}_t = \phi(Vh_t + b_y),$$

with fixed matrices and biases U, W, V, b_h, b_y. Increasing the sequence length from T to

$T+1$ adds one extra application of these same equations at time $T+1$, but introduces no new trainable variables. Therefore, longer sequences increase computation depth in time, not model size. $\square$

This property allows RNNs to model temporal dependencies over varying horizons while maintaining a compact parametrisation. Recurrent Neural Networks (RNNs) are trained using *Backpropagation Through Time* (BPTT) [2].

5 Training and Optimisation Challenges

5.1 Vanishing and Exploding Gradients

For a sequence of length T, define the per-step loss

$$\ell_t := \ell(\hat{y}_t, y_t), \qquad \mathcal{L} = \sum_{t=1}^{T} \ell_t,$$

where y_t denotes the observed gold price at time t.

We use the column-vector gradient convention: for a scalar function f and a vector x, $\nabla_x f \in \mathbb{R}^{\dim(x)}$. Let

$$a_t := U y_t + W h_{t-1} + b_h, \qquad h_t = \phi(a_t),$$

and define the BPTT adjoint at time k by $\delta_k := \nabla_{h_k} \mathcal{L}$.

For each $i = 2, \ldots, T$, define the recurrence Jacobian

$$J_i := \frac{\partial h_i}{\partial h_{i-1}} = \operatorname{diag}\big(\phi'(a_i)\big)\, W, \qquad J_i^\top = W^\top \operatorname{diag}\big(\phi'(a_i)\big)\,.$$

Then

$$\delta_k = \sum_{t=k}^{T} \nabla_{h_k} \ell_t = \sum_{t=k}^{T} \left(\prod_{i=k+1}^{t} J_i^\top \right) \nabla_{h_t} \ell_t,$$

with the empty product (when $t = k$) equal to the identity. As a result, gradients involve repeated matrix products across time, making optimisation highly sensitive to sequence length.

Let $\lambda_{\max}$ denote the largest singular value of W and assume the activation function satisfies

$$|\phi'(\cdot)| \le c \le 1,$$

which covers common cases such as sigmoid ($c < 1$) and tanh (where the tight bound is $c = 1$). This finite upper bound keeps each recurrent Jacobian uniformly controlled across

time, allowing the repeated Jacobian product to be bounded:

$$\left\| \prod_{i=k+1}^{t} \frac{\partial h_i}{\partial h_{i-1}} \right\| \leq (\lambda_{\max} c)^{t-k}.$$

If $\lambda_{\max} c < 1$, gradients decay exponentially as $t - k$ increases,

$$(\lambda_{\max} c)^{t-k} \longrightarrow 0,$$

leading to the *vanishing gradient problem*. In this regime, error signals propagated back to early time steps become extremely small, so the parameters associated with those steps receive negligible updates. Consequently, the model struggles to learn long-range temporal dependencies present in historical gold price dynamics.

If $\lambda_{\max} c = 1$, the bound is neutral (order one): it does not force exponential decay or growth. In this boundary regime, gradient norms may remain roughly stable but can still vary due to the detailed structure of the Jacobian products.

If $\lambda_{\max} c > 1$, gradient norms grows exponentially,

$$(\lambda_{\max} c)^{t-k} \longrightarrow \infty,$$

resulting in the *exploding gradient problem*. This causes unstable parameter updates and numerical divergence during training.

Both phenomena represent fundamental optimisation challenges in training RNNs. These limitations motivate the use of gated architectures, such as Long Short-Term Memory (LSTM) networks, which are explicitly designed to regulate information flow and preserve gradients over long time horizons [9].

5.2 Fitting, underfitting and overfitting

Given a parametric recurrent model $\hat{f}_\theta$, training aims to minimise the empirical sequence loss over the observed series. In this setting, underfitting occurs when the model fails to capture the underlying temporal structure, whereas overfitting occurs when it fits the training data too closely and generalises poorly to unseen observations.

5.2.1 Underfitting

Underfitting arises when the effective capacity of the recurrent model is insufficient to approximate the underlying temporal dynamics of the series. In practice, this occurs when gradient contributions from distant time steps vanish,

$$\left\| \frac{\partial \mathcal{L}}{\partial h_k} \right\| \approx 0 \quad \text{for } k \ll T,$$

preventing parameter updates from reflecting long-range dependencies. Although the nominal model may contain many parameters, vanishing gradients reduce the *effective depth in time*, resulting in systematically biased predictions and persistent forecasting error across both training and validation windows.

5.2.2 Overfitting

Overfitting happens when the recurrent model effectively memorizes the unique patterns in the observed price history. In this situation, gradients stay stable or increase over short time periods, enabling the model to closely follow the training sequence,

$$\ell(\hat{y}_t, y_t) \approx 0 \quad \text{for } t \in \{1, \dots, T\},$$

while failing to generalise to unseen future intervals. This behaviour is exacerbated when the number of trainable parameters is large, relative to the effective sample size and when regularisation is absent.

5.2.3 Regularised fitting

To control instability and improve generalisation, we modify the empirical loss by adding an L^2 penalty on the parameter vector, yielding the regularised objective

$$J(\theta) = \mathcal{L}(\theta) + \frac{\lambda}{2}\|\theta\|_2^2, \qquad \lambda > 0.$$

Theorem 5.1 (Norm control under L^2 regularisation)**.** *Assume $\mathcal{L}(\theta) \geq 0$ for all θ and let*

$$\theta^* \in \arg\min_{\theta} J(\theta).$$

Then

$$\|\theta^*\|_2^2 \leq \frac{2}{\lambda}\,\mathcal{L}(0).$$

In particular, the minimiser of the regularised problem has bounded parameter norm.

Proof. By optimality of θ^*, we have

$$J(\theta^*) \leq J(0) = \mathcal{L}(0).$$

Hence

$$\mathcal{L}(\theta^*) + \frac{\lambda}{2}\|\theta^*\|_2^2 \leq \mathcal{L}(0).$$

Using $\mathcal{L}(\theta^*) \geq 0$ gives

$$\frac{\lambda}{2}\|\theta^*\|_2^2 \leq \mathcal{L}(0),$$

which implies

$$\|\theta^*\|_2^2 \leq \frac{2}{\lambda}\,\mathcal{L}(0).$$

Therefore L^2 regularisation explicitly prevents arbitrarily large weights. $\square$

To control exploding gradients, it is desirable to impose an explicit bound on the magnitude of each optimisation step. This motivates the use of gradient clipping.

Theorem 5.2 (Gradient clipping bounds update magnitude)**.** *Consider the gradient step*

$$\theta_{t+1} = \theta_t - \eta\, \tilde{g}_t,$$

with clipped gradient

$$\tilde{g}_t = \begin{cases} g_t, & \|g_t\|_2 \le \tau, \\ \tau \dfrac{g_t}{\|g_t\|_2}, & \|g_t\|_2 > \tau, \end{cases} \qquad g_t = \nabla_\theta \mathcal{L}(\theta_t).$$

Then

$$\|\theta_{t+1} - \theta_t\|_2 \le \eta\tau,$$

for every iteration t.

Proof. From the update rule,

$$\|\theta_{t+1} - \theta_t\|_2 = \eta\, \|\tilde{g}_t\|_2.$$

If $\|g_t\|_2 \le \tau$, then $\tilde{g}_t = g_t$ and $\|\tilde{g}_t\|_2 \le \tau$. If $\|g_t\|_2 > \tau$, then

$$\|\tilde{g}_t\|_2 = \left\| \tau \frac{g_t}{\|g_t\|_2} \right\|_2 = \tau.$$

So in all cases $\|\tilde{g}_t\|_2 \le \tau$, yielding

$$\|\theta_{t+1} - \theta_t\|_2 \le \eta\tau.$$

Thus clipping prevents single-step exploding updates during BPTT. $\square$

Theorem 5.3 (Early stopping as model selection over epochs)**.** *Let θ_t be parameters after epoch t and define*

$$t^* \in \arg\min_{1 \le t \le T_{\max}} \mathcal{L}_{\text{val}}(\theta_t).$$

Choosing $\hat{\theta} = \theta_{t^*}$ guarantees

$$\mathcal{L}_{\text{val}}(\hat{\theta}) = \min_{1 \le t \le T_{\max}} \mathcal{L}_{\text{val}}(\theta_t) \le \mathcal{L}_{\text{val}}(\theta_{T_{\max}}).$$

Proof. The set $\{1, \ldots, T_{\max}\}$ is finite, so a minimiser t^* exists. By definition of minimum,

$$\mathcal{L}_{\text{val}}(\theta_{t^*}) \le \mathcal{L}_{\text{val}}(\theta_t) \quad \text{for all } t \in \{1, \ldots, T_{\max}\}.$$

Taking $t = T_{\max}$ gives the stated inequality. Hence selecting the best validation epoch cannot perform worse (on validation loss) than training until the final epoch. $\square$

6 LSTM

While conventional RNNs rely on a single hidden state updated at each time step, Long Short-Term Memory (LSTM) networks offer a more organized memory system.

6.1 LSTM Architecture

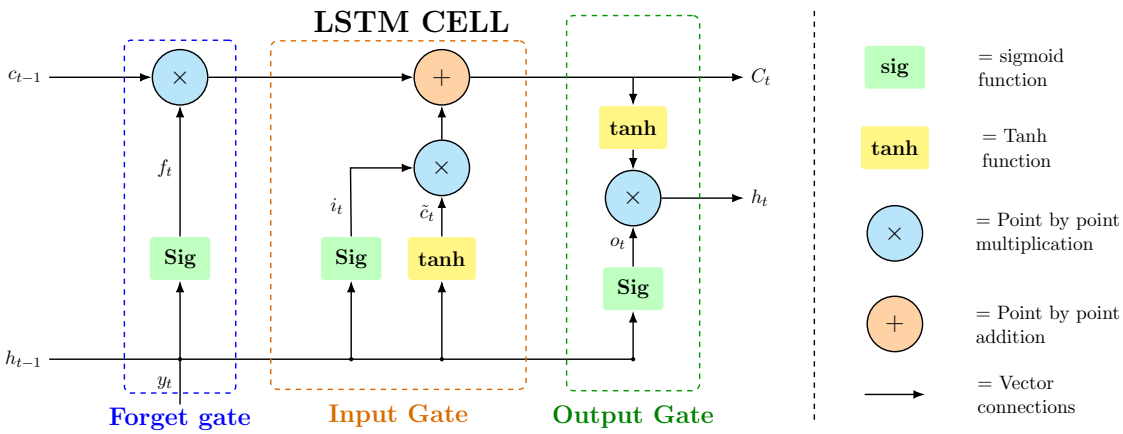

Figure 3: Schematic LSTM cell architecture showing the interaction between forget, input and output gates with the cell and hidden states.

This design helps keep important information over longer periods. It also tackles the vanishing gradient problem by adjusting the hidden state with an explicit memory cell that stores long-term information and is regulated by a set of gating mechanisms.

Definition 6.1 (LSTM State Variables)**.** *At each time step t, an LSTM maintains:*

- *a cell state $c_t \in \mathbb{R}^{d_h}$, representing the long-term memory and*
- *a hidden state $h_t \in \mathbb{R}^{d_h}$, representing the working (short-term) memory.*

As visible in Figure 3, these states evolve through the action of three gates: the forget gate, the input gate and the output gate.

The first mechanism determines which information should be retained from the previous memory state.

Definition 6.2 (LSTM Forget Gate)**.** *The forget gate is defined as*

$$f_t = \sigma\left(W_f[y_t, h_{t-1}] + b_f\right),$$

where $[y_t, h_{t-1}]$ denotes the concatenation of the current input and previous hidden state, $W_f \in \mathbb{R}^{d_h \times (d_y + d_h)}$ is the forget gate weight matrix, $b_f \in \mathbb{R}^{d_h}$ is the bias vector and σ is

the sigmoid activation function, also d_y denotes the dimension of the input (which we will later see is 4).

Since σ maps to the interval $[0, 1]$, each component of f_t determines the extent to which information from the previous cell state is retained or discarded.

Lemma 6.3. *If $(f_t)_i = 0$, then the i-th component of the previous cell state c_{t-1} is completely forgotten. If $(f_t)_i = 1$, the corresponding information is fully retained.*

The update of the cell state involves the Hadamard product $f_t \odot c_{t-1}$. Thus, a zero entry nullifies the corresponding component, while a unit entry preserves it unchanged. New information is then incorporated into the memory through the input gate.

Definition 6.4 (LSTM Input Gate and Candidate Cell State)**.** *The input gate and candidate cell state are given by*

$$i_t = \sigma\left(W_i[y_t, h_{t-1}] + b_i\right), \qquad \hat{c}_t = \tanh\left(W_c[y_t, h_{t-1}] + b_c\right),$$

where $W_i, W_c \in \mathbb{R}^{d_h \times (d_y + d_h)}$ and $b_i, b_c \in \mathbb{R}^{d_h}$ are the respective weight matrices and bias vectors.

The candidate cell state $\hat{c}_t \in [-1, 1]^{d_h}$ represents new information that may be written to memory.

Definition 6.5 (Cell State Update)**.** *The cell state is updated according to*

$$c_t = f_t \odot c_{t-1} + i_t \odot \hat{c}_t,$$

where $\odot$ denotes the element-wise (Hadamard) product.

The cell state is central to the stability of LSTMs. Unlike standard RNNs, where repeated multiplication by recurrent weight matrices may lead to rapid gradient decay, the LSTM cell state is updated additively. Consequently, when the forget gate activations remain sufficiently close to one, information and gradients are preserved more effectively over long time horizons.

Finally, the output gate controls how much of the internal memory is exposed to the next layer.

Definition 6.6 (Output Gate and Hidden State)**.** *The output gate and hidden state are defined by*

$$o_t = \sigma\left(W_o[y_t, h_{t-1}] + b_o\right), \qquad h_t = o_t \odot \tanh(c_t),$$

where $W_o \in \mathbb{R}^{d_h \times (d_y + d_h)}$ and $b_o \in \mathbb{R}^{d_h}$ are the output gate parameters.

The hidden state thus represents a gated, non-linear transformation of the cell state and serves as the LSTM's output at each time step.

6.2 Data pre-processing

The gold price time series consists of daily closing prices from 1 January 2020 to 1 November 2025. To capture temporal patterns at different horizons, the full series is partitioned into overlapping intervals of fixed length 30 days.

Each interval is independently normalised to the range $[0, 1]$ using min-max scaling. This normalisation ensures numerical stability during training and allows the model to focus on relative price movements rather than absolute price levels.

Finally, the dataset is split chronologically, with 80% of the intervals used for training and the remaining 20% reserved for testing, ensuring that model evaluation is performed on unseen future data.

6.2.1 Feature Implementation

To enhance the predictive accuracy with the information available to the model, several additional features are derived from the price series and from broader market data.

Definition 6.7 (Moving Averages)**.** *Let $\{y_t\}_{t\geq 0}$ denote the daily gold closing price series. The 7-day and 30-day moving averages at time t are defined by*

$$MA_{7,t} := \frac{1}{7}\sum_{i=t-6}^{t} y_i, \qquad MA_{30,t} := \frac{1}{30}\sum_{i=t-29}^{t} y_i.$$

These quantities capture short-term and medium-term trend information, respectively.

Definition 6.8 (Log Return)**.** *The log return at time t is defined as*

$$r_t := \log(y_t) - \log(y_{t-1}),$$

which represents the relative daily price change and is commonly used to induce stationarity in financial time series.

Definition 6.9 (Daily Feature Vector)**.** *Each trading day t is represented by the feature vector*

$$\mathbf{y}_t := \big(y_t, MA_{7,t}, MA_{30,t}, r_t\big) \in \mathbb{R}^4,$$

where each components is min-max scaled to the interval $[0, 1]$ before the construction of input sequences.

6.2.2 Sequence Construction

The LSTM model is trained using overlapping input sequences of 30 consecutive days, allowing the model to predict the next day's price based on the previous 30 days of data. Let T denote the total number of daily observations. For each $i = 1, \ldots, T - 30$, we define

the input sequence

$$\mathbf{Y}_i = \big(\mathbf{y}_i, \mathbf{y}_{i+1}, \ldots, \mathbf{y}_{i+29}\big).$$

6.2.3 Prediction Targets

The target associated with each input sequence $\mathbf{Y}_i$ is the closing price on the following day:

$$y_{i+30}.$$

Since the training data associated with the target output would be incomplete if we used any values prior to y_{31}, the full target vector used for training is

$$\bar{\mathbf{y}} = \big(y_{31}, y_{32}, \ldots, y_T\big),$$

and the model produces the corresponding predictions

$$\hat{\mathbf{y}} = \big(\hat{y}_{31}, \hat{y}_{32}, \ldots, \hat{y}_T\big).$$

6.3 Loss Function Comparison

Financial market data exhibit rapid regime changes, volatility clustering and heavy tails. For this reason, we compare multiple candidate loss functions under the same model architecture and training setup, then choose a single objective for the final model.

Definition 6.10 (Mean Squared Error)**.** *Given true values $\{y_i\}_{i=1}^N$ and predictions $\{\hat{y}_i\}_{i=1}^N$, the Mean Squared Error (MSE) is defined as*

$$\mathcal{L}_{MSE}(y, \hat{y}) = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2.$$

MSE provides smooth, differentiable gradients and is straightforward to optimise using gradient-based methods. However, due to its quadratic penalisation, it is highly sensitive to outliers and tends to produce overly smoothed predictions, often underestimating sharp market movements.

Definition 6.11 (Huber Loss)**.** *For a threshold parameter $\delta > 0$, the Huber loss is defined as*

$$\mathcal{L}_\delta(y, \hat{y}) = \begin{cases} \frac{1}{2}(y - \hat{y})^2, & \text{if } |y - \hat{y}| \le \delta, \\ \delta\left(|y - \hat{y}| - \frac{1}{2}\delta\right), & \text{if } |y - \hat{y}| > \delta. \end{cases}$$

The Huber loss behaves quadratically for small errors and linearly for large errors, making it more robust to outliers than MSE while retaining stable gradients. This makes it well suited for financial time series with occasional sharp price changes, although it does not explicitly prioritise extreme upward or downward spikes.

Definition 6.12 (Quantile Loss)**.** *For a quantile level $q \in (0,1)$, the quantile loss is defined as*

$$\mathcal{L}_q(y, \hat{y}) = \max\left(q(y - \hat{y}),\ (q-1)(y - \hat{y})\right).$$

Quantile loss estimates conditional quantiles rather than conditional means. Choosing an upper quantile (e.g. $q = 0.9$) places greater penalty on under-prediction, making it particularly effective for capturing extreme upward price movements and asymmetric risk.

After comparing the experimental results from MSE, Huber loss and quantile loss, we found that MSE gives the strongest overall performance for this task. In particular, it provides the best balance between optimisation stability and price-level prediction accuracy on the validation and test splits. Therefore, the final LSTM configuration in this report is trained using MSE as the single selected loss function.

6.4 Layers

6.4.1 Architecture and Layer Design

Multiple layers are used to model our time-series data, each layer of the network extracts features at different levels of abstraction.

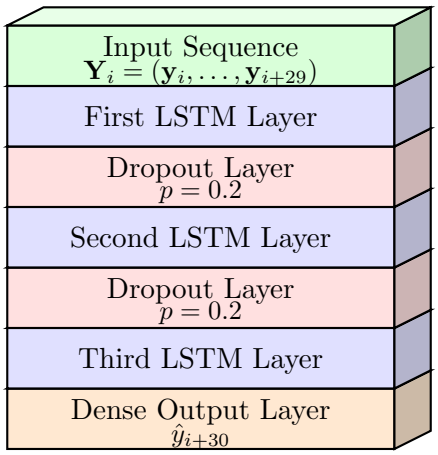

Figure 4: Stacked architecture of the proposed LSTM forecasting model, shown from input at the top to output at the bottom.

Layer Composition The model consists of three LSTM layers followed by a Dense output layer. The rationale behind this choice is as follows:

- **First LSTM layer:** Extracts low-level features, such as short-term fluctuations and basic temporal patterns.
- **Second LSTM layer:** Learns intermediate patterns by combining low-level features into more complex sequences.

- **Third LSTM layer:** Captures high-level dependencies and longer-term trends, allowing the model to understand overarching structures in the data.

6.4.2 Dropout Layers

Dropout is a regularisation technique in which each hidden unit is independently deactivated with probability p during training. This reduces co-adaptation between neurons and improves generalisation in deep recurrent stacks. In our model, we use $p = 0.2$ between LSTM layers [11].

Let $h_t^l \in \mathbb{R}^{d_l}$ denote the activation of the lth hidden layer. A binary mask $r^l \in \{0,1\}^{d_l}$ is sampled such that

$$r_i^l \sim \text{Bernoulli}(1-p),$$

where p denotes the dropout rate. The output of the dropout layer during training is then given by

$$\tilde{h}_t^l = \frac{r^l \odot h_t^l}{1-p},$$

where $\odot$ denotes element-wise multiplication. The scaling factor $(1-p)^{-1}$ ensures that the expected magnitude of the activations remains unchanged.

During inference, dropout is disabled and all neurons remain active, allowing the full capacity of the network to be used for prediction. In this time-series branch, dropout is applied after the first and second LSTM layers to reduce overfitting on noisy financial price dynamics.

6.4.3 Dense Output Layer

A Dense (fully connected) output layer maps the final hidden state of the last LSTM layer to the target prediction. For a single-step forecasting task, the output is given by

$$\hat{y}_T = V h_T + b_y,$$

where h_T is the hidden state produced by the final LSTM layer, $V \in \mathbb{R}^{1 \times d_h}$ is the output weight matrix and $b_y \in \mathbb{R}$ is the bias term.

This layer transforms the high-level temporal representation learned by the stacked LSTM layers into a single scalar prediction corresponding to the next-day gold price.

6.5 Training Procedure and Model Configuration

Training is carried out using mini-batch stochastic optimisation with backpropagation through time. At each iteration, the model processes a batch of input sequences, computes the loss between predictions and targets and updates the parameters using the Adam

optimiser. Training proceeds over multiple epochs, while validation loss is monitored to assess generalisation and detect overfitting.

Definition 6.13 (Batch Size)**.** *The* batch size B *is the number of training samples used to compute a single parameter update.*

Let $\ell(y_i, \hat{y}_i; \theta)$ denote the loss on a single training sample, where $\theta \in \mathbb{R}^d$ is the parameter vector.

Definition 6.14 (Mini-batch Gradient)**.** *Given a batch* $\mathcal{B} = \{(y_i, \hat{y}_i)\}_{i=1}^{B}$*, the mini-batch gradient is*

$$\nabla_\theta \mathcal{L}_\mathcal{B}(\theta) := \frac{1}{B} \sum_{i=1}^{B} \nabla_\theta \ell(y_i, \hat{y}_i; \theta).$$

Definition 6.15 (Epoch)**.** *An* epoch *is one complete pass through the full training dataset.*

If mini-batches of size B are used, one epoch consists of $\lceil N/B \rceil$ update steps for a training set of size N.

Definition 6.16 (Validation Loss)**.** *Given a validation dataset* $\{(y_i, \hat{y}_i)\}_{i=1}^{M}$*, the validation loss is*

$$\mathcal{L}_{\text{val}}(\theta) := \frac{1}{M} \sum_{i=1}^{M} \ell(y_i, \hat{y}_i; \theta).$$

Monitoring validation loss during training provides a practical criterion for assessing out-of-sample performance and detecting overfitting.

Adam is an adaptive first-order optimiser that we use because it combines momentum with per-parameter learning-rate scaling, which helps stabilise and accelerate training on noisy mini-batch gradients [10].

Definition 6.17 (Adam Optimiser)**.** *Let* $g_t := \nabla_\theta \mathcal{L}(\theta_t)$ *denote the stochastic gradient at iteration* t*. Adam updates exponentially weighted estimates of the first and second moments:*

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \qquad v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2.$$

The bias-corrected moments are

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \qquad \hat{v}_t = \frac{v_t}{1 - \beta_2^t},$$

and the parameter update is

$$\theta_{t+1} = \theta_t - \alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon}.$$

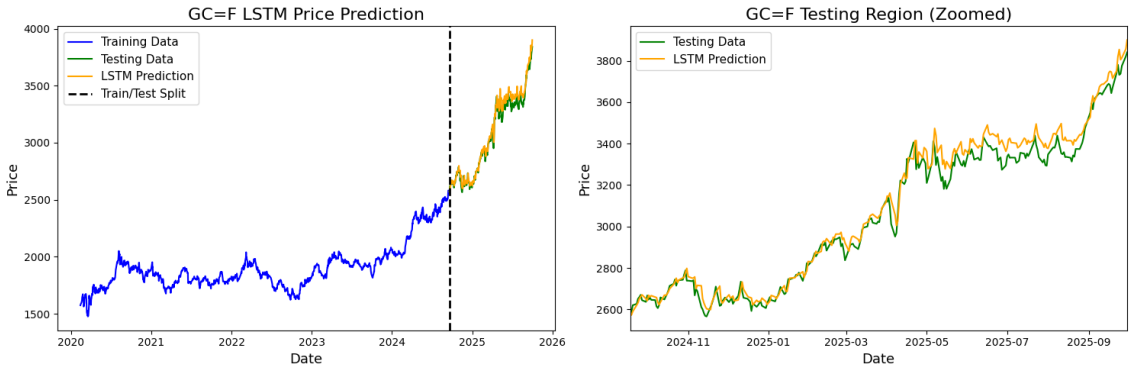

Figure 5: Time-series model prediction versus observed gold prices under a chronological 80%/20% train-test split.

6.6 Analysis of Prediction

Figure 5 shows that the baseline LSTM captures the overall upward movement in gold prices reasonably well, with the predicted series closely following the observed test data after the train-test split. However, the forecast remains smoother than the true series and is less responsive during short-term fluctuations and sharper upward moves, indicating some difficulty adapting to sudden changes in market conditions.

The model was trained for 25 epochs, after which validation performance had largely stabilised and further training increased the risk of overfitting. On the baseline time-series-only setup, the model achieves RMSE = 53.9361, $R^2 = 0.9755$, Return MSE = 0.000259 and DA = 48.46% on the test set. These results show strong price-level tracking overall, although the $< 50\%$ Directional accuracy suggests that directional prediction remains weak.

To address this limitation, the next stage incorporates news information through sentiment modelling. Rather than using a DNN to treat headline embeddings as static features, the final approach adopts a Mamba state space model, which is better suited to capturing how sentiment evolves over time. This allows the sentiment component to model temporal dependencies in the news stream more effectively before being integrated with the price-based forecasting pipeline [13].

7 Sentiment-Driven Price Prediction

This section describes how headlines are turned into sentiment signals for the hybrid forecast. For each headline $\mathcal{H}_t$ observed on day t, we tokenise the text, encode it with FinBERT to obtain a contextual embedding, filter out weakly relevant items and combine that embedding with the same-day price input before passing it to a Mamba-based regressor to predict the next-step price gradient $\Delta y_t = y_{t+1} - y_t$. We then aggregate headline-level predictions into one daily sentiment signal and combine it with the price-

only time-series forecast. This lets the framework use information from news that is not captured by historical prices alone.

7.1 Tokenisation

Neural networks require numerical inputs, so each headline $\mathcal{H}_t$ must first be converted into token identifiers before it can be passed to FinBERT. FinBERT, like BERT, uses the WordPiece tokeniser [6, 7], which segments words into subword pieces that balance vocabulary size and coverage. For example, the sentence "The company reported strong quarterly earnings." becomes the token sequence

$$\mathcal{H}_t = [\text{the, company, reported, strong, quarter, ly, earnings.}]$$

The decomposition of "quarterly" into "quarter" and "ly" highlights the distinction between full words and subwords.

7.1.1 Token Formulation

Definition 7.1 (Tokeniser)**.** *Let the vocabulary be denoted by*

$$\mathcal{V} = \{\tau_1, \tau_2, \ldots, \tau_{|\mathcal{V}|}\}, \quad \tau_i \in \{1, 2, \ldots, |\mathcal{V}|\}.$$

A tokeniser is a mapping

$$\Psi : \text{Text} \to \{1, 2, \ldots, |\mathcal{V}|\}^n, \qquad S_t = \Psi(\mathcal{H}_t) = (\tau_{t,1}, \ldots, \tau_{t,n}),$$

which maps a text sequence to a sequence of integer token identifiers corresponding to subwords, where n is the number of subword tokens.

Proposition 7.2 (WordPiece Tokenisation)**.** *Let w be a word and let $\mathcal{V}$ denote the WordPiece vocabulary. WordPiece tokenisation decomposes w into a sequence of sub-tokens $(\tau_1, \ldots, \tau_k)$ with each $\tau_j \in \mathcal{V}$ such that concatenating the sub-tokens reconstructs w. In practice, this decomposition is implemented via longest-match-first subword segmentation over the vocabulary [4].*

7.1.2 Padding and Truncation

Transformer-based models require sequences of fixed length L. A tokenised sequence $S = (\tau_{t,1}, \ldots, \tau_{t,n})$ is therefore padded or truncated according to

$$S_t^{\text{pad}} = \begin{cases} (\tau_{t,1}, \ldots, \tau_{t,n}, 0, \ldots, 0) & n < L \\ (\tau_{t,1}, \ldots, \tau_{t,L}) & n \geq L, \end{cases}$$

where 0 denotes the padding token identifier.

Proposition 7.3 (Fixed-Length Representation)**.** *Padding and truncation ensure that all tokenised sequences lie in* $\{0, 1, \ldots, |\mathcal{V}|\}^L$*, enabling efficient batch processing and consistent tensor dimensions within the transformer architecture.*

7.2 Embeddings and the Transformer Encoder

After tokenisation and length normalisation, each sequence is mapped into continuous vectors before entering the transformer encoder. FinBERT uses three embedding matrices: token embeddings E_t, segment embeddings E_s and positional embeddings E_p. The hidden layer size for FinBERT is fixed at $H = 768$, hence

$$E_t \in \mathbb{R}^{|\mathcal{V}| \times H}, \qquad E_s \in \mathbb{R}^{2 \times H}, \qquad E_p \in \mathbb{R}^{L \times H}.$$

For a token sequence, each token position i is associated with:

- $\tau_{t,i}$: the token identifier,
- $s_{t,i} \in \{0, 1\}$: the segment identifier,
- $p_i \in \{0, \ldots, L-1\}$: the token position.

The embedding for token i is computed as

$$\xi_{t,i} = E_t[\tau_{t,i}] + E_s[s_{t,i}] + E_p[p_i] \in \mathbb{R}^H,$$

yielding the embedded sequence

$$X_t = (\xi_{t,1}, \ldots, \xi_{t,L}) \in \mathbb{R}^{L \times H}.$$

7.2.1 Role of the Transformer Encoder

The transformer encoder used in this work is pre-trained FinBERT and is kept fixed during model development. Its role is to map the embedded token sequence X_t into a contextual representation that captures semantic and financial relationships between tokens.

Formally, the encoder defines a mapping

$$T_\theta : \mathbb{R}^{L \times H} \to \mathbb{R}^{L \times H},$$

where θ denotes the fixed pre-trained parameters. The output

$$Z_t = T_\theta(X_t)$$

is treated as a deterministic feature representation of the input headline.

7.2.2 Self-Attention Mechanism

The core operation within the transformer is scaled dot-product attention.

Definition 7.4 (Scaled Dot-Product Attention)**.** *Let $Q, K, V \in \mathbb{R}^{L \times d_k}$ denote the query, key and value matrices. The attention operator is defined by*

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V.$$

The matrix $QK^\top$ contains pairwise similarity scores between tokens, while the normalisation by $\sqrt{d_k}$ stabilises the scale of these inner products during training.

Lemma 7.5 (Normalisation of Attention Weights)**.** *Each row of*

$$\text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right),$$

forms a probability vector. In particular, the attention weights are non-negative and sum to one across all token positions.

This ensures that every token is updated as a convex combination of the value vectors, with coefficients determined by contextual relevance, i.e. how much **attention** to give it.

7.2.3 Multi-Head Attention

Rather than using a single attention map, transformers employ multi-head attention, which allows the model to attend simultaneously to information from distinct representation subspaces.

Definition 7.6 (Multi-Head Attention)**.** *For h attention heads, define*

$$\text{head}_j = \text{Attention}(X_t W_j^Q, X_t W_j^K, X_t W_j^V), \qquad j = 1, \ldots, h,$$

where W_j^Q, W_j^K, W_j^V are learned projection matrices. Multi-head attention is then given by

$$\text{MHA}(X_t) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h) W^O,$$

where W^O is an output projection matrix [4].

Consequently, different heads may specialise in different linguistic or financial relationships, such as negation, emphasis, causality or entity-specific interactions.

7.2.4 Transformer Encoder Layer

A transformer encoder layer has two main steps: multi-head self-attention and a feed-forward block applied at each token position. Both steps use a residual connection followed

by layer normalisation [5]. If Z denotes the input to the layer, the computations are

$$Z' = \text{LayerNorm}(Z + \text{MHA}(Z)),$$

$$Z'' = \text{LayerNorm}(Z' + \text{FFN}(Z')).$$

Definition 7.7 (Position-Wise Feed-Forward Network)**.** *The feed-forward network within the transformer is defined by*

$$\text{FFN}(x) = \max\{0, xW_1 + b_1\}W_2 + b_2,$$

where W_1, W_2 and b_1, b_2 are learned parameters and the non-linearity is applied componentwise.

Lemma 7.8 (Local Transformation Property)**.** *The feed-forward network acts independently on each token position. If $Z = (Z_1, \ldots, Z_L)$, then*

$$\text{FFN}(Z)_i = \text{FFN}(Z_i), \qquad i = 1, \ldots, L.$$

In practical terms, self-attention lets each token use information from other tokens in the same headline (for example, linking "misses" with "earnings" or "guidance"). The feed-forward network then processes each token separately to sharpen these contextual signals into features that are easier for the downstream sentiment model to use.

7.2.5 Transformer Encoder and Pooling

After passing through the transformer encoder, the embedded sequence becomes

$$Z_t = T_\theta(X_t) \in \mathbb{R}^{L \times H}.$$

A fixed-length headline representation is then obtained from the special classification token:

$$H_t = Z_{t,[CLS]} \in \mathbb{R}^H.$$

This pooled vector summarises the headline and is the feature passed to the downstream sentiment branch.

7.2.6 FinBERT as a Domain-Specific Transformer

FinBERT is a domain-adapted version of BERT trained on financial text. The purpose of this additional pre-training is to align the representation space with the language used in earnings reports, analyst commentary, market headlines and macroeconomic news.

Lemma 7.9 (Domain Adaptation of FinBERT)**.** *Let θ denote the parameters of the orig-*

inal BERT model and let $\mathcal{D}_{\text{finance}}$ denote a financial text corpus. FinBERT parameters may be viewed as the solution of an optimisation problem of the form

$$\theta_{\text{FinBERT}} = \arg\min_{\theta} \mathbb{E}_{(X,y)\sim\mathcal{D}_{\text{finance}}} \ell(f_\theta(X), y),$$

where ℓ is the pre-training or fine-tuning objective relevant to the financial domain.

Remark 7.10. *This domain adaptation improves the encoding of financially meaningful phrases such as "beats expectations", "misses guidance", "hawkish stance" or "risk-off sentiment", which may carry predictive significance for short-term market behaviour.*

7.2.7 Relevance Filtering

Before regression, we apply a relevance filter to the pooled embedding. A sentiment score is derived according to

$$s_t = g_\phi(H_t),$$

where g_ϕ is a pre-trained feed-forward classifier mapping embeddings to scalar sentiment values. To reduce noise from weakly informative or irrelevant headlines, a filtering mechanism is applied, retaining only embeddings whose sentiment magnitude exceeds a threshold $\vartheta = 0.6$. Formally,

$$\tilde{H}_t = \begin{cases} H_t, & |s_t| \geq \vartheta, \\ 0, & |s_t| < \vartheta. \end{cases}$$

This stage acts as a coarse gate: only sufficiently informative embeddings are propagated to the predictor, producing the filtered representation $\tilde{H}_t$ used in the next stage.

7.3 Input Structure and Data Preparation

The downstream model uses both headline and price context. Let

$$\tilde{z}_t = [\tilde{H}_t \,;\, \tilde{y}_t] \in \mathbb{R}^{769},$$

where $\tilde{y}_t$ denotes the standardised observed price for day t. The predictor maps $\tilde{z}_t$ to the next-step future gradient,

$$\Delta y_t = y_{t+1} - y_t,$$

framing sentiment learning as regression on one-step price changes rather than absolute price level.

7.4 Mamba State Space Model

During initial development, a dense neural network was used to map the headline embedding directly to a scalar prediction. While such a mapping is effective for static nonlinear regression, it does not explicitly introduce an internal state capable of learning structured

dependencies within the embedding representation. Figure 6 summarises the architecture of the Mamba sentiment branch used in this work.

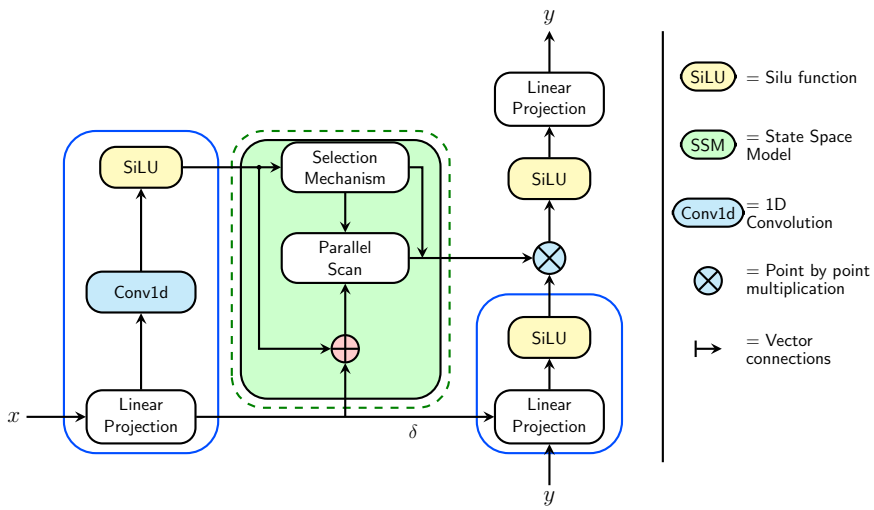

Figure 6: Mamba architecture used for the sentiment branch, showing the selective state space block and gated pathways from input embedding to prediction output.

7.4.1 Continuous and Discrete State Space Formulation

Definition 7.11 (Linear State Space Model)**.** *A continuous-time state space model is defined by*

$$\frac{d}{dt}x(t) = Ax(t) + Bu(t), \qquad y(t) = Cx(t),$$

where $x(t) \in \mathbb{R}^d$ is the hidden state, $u(t) \in \mathbb{R}^m$ is the input and $A \in \mathbb{R}^{d\times d}$, $B \in \mathbb{R}^{d\times m}$, $C \in \mathbb{R}^{r\times d}$ are parameter matrices. In practical terms, $y(t)$ (and its discrete counterpart y_t) is the observable readout signal produced by projecting the latent state through C; in this section it represents the model output used for prediction, not the raw market price level.

After discretisation, the model takes the form

$$x_t = \bar{A}x_{t-1} + \bar{B}u_t, \qquad y_t = Cx_t,$$

where t indexes the discrete sequence positions.

Lemma 7.12 (Recursive Expansion of the State)**.** *The discrete state satisfies*

$$x_t = \sum_{k=1}^{t} \bar{A}^{t-k}\bar{B}u_k + \bar{A}^t x_0.$$

In particular, if $x_0 = 0$, then the current state is a weighted sum of all previous inputs.

This identity shows that state space models possess an intrinsic notion of memory: past

inputs continue to influence the present state through repeated multiplication by the transition operator.

7.4.2 Selective State Space Dynamics

Classical state space models are time-invariant, meaning the same matrices govern the update at every step. Mamba strengthens this framework by allowing the dynamics to depend on the current input.

Definition 7.13 (Selective State Space Model)**.** *Let $u_t \in \mathbb{R}^m$ denote the input at step t. A selective state space model is defined by*

$$x_t = A(u_t)x_{t-1} + B(u_t)u_t, \qquad y_t = C(u_t)x_t,$$

where $A(\cdot)$, $B(\cdot)$ and $C(\cdot)$ are learned input-dependent transformations.

Lemma 7.14 (Adaptive Memory Filtering)**.** *The input dependence of $A(u_t)$ allows the model to retain, suppress or amplify previously stored information according to the content of the current input.*

This is the key structural difference between Mamba and an LSTM network. In an LSTM, the embedding is processed through gated recurrent updates of hidden and cell states. In Mamba, the embedding is filtered through a latent state whose evolution is controlled by input-dependent state space operators.

7.4.3 Comparison with LSTM Networks

Proposition 7.15 (Structural Difference Between LSTM and Mamba Models)**.** *Let f_{lstm} denote an LSTM predictor and f_{mamba} denote a Mamba predictor. Then*

$$f_{\text{lstm}}(\tilde{z}_t) = g(\tilde{z}_t, h_{t-1}, c_{t-1}), \qquad f_{\text{mamba}}(\tilde{z}_t) = g(\tilde{z}_t, x_{t-1}),$$

where (h_{t-1}, c_{t-1}) are the LSTM hidden and cell states and x_{t-1} is the Mamba state variable. Both models are recursive, but LSTM relies on explicit gate-wise recurrent updates, whereas Mamba uses input-conditioned state space dynamics.

7.4.4 Convolutional Interpretation and Complexity

State space models also admit a convolutional interpretation. Under suitable regularity assumptions, the output can be written as

$$y_t = \sum_{k=1}^{t} K_{t-k} u_k,$$

where the kernel K_{t-k} is induced by the underlying state dynamics.

Theorem 7.16 (Linear-Time Evaluation)**.** *For an input sequence of length T, the selective state space model can be evaluated in $\mathcal{O}(T)$ time.*

This contrasts with transformer self-attention, whose quadratic complexity in the sequence length is $\mathcal{O}(T^2)$. Although headlines in this project are short, the linear scaling of Mamba becomes advantageous when the model is extended to longer text windows or higher-frequency text streams.

7.4.5 Gated State Update

To stabilise optimisation and regulate information flow, a gated state update is introduced.

Definition 7.17 (Gated State Update)**.** *Let G be a learned gating matrix. The hidden state evolves according to*

$$x_t = \sigma(Gu_t) \odot (\bar{A}x_{t-1}) + \big(1 - \sigma(Gu_t)\big) \odot (\bar{B}u_t),$$

where σ is the sigmoid function and $\odot$ denotes the Hadamard product.

This update has the same qualitative purpose as gating in LSTMs: it controls how much prior state information is preserved and how much new information is injected. The difference is that the gating is applied within a state space framework rather than within a recurrent cell with separate forget, input and output gates.

7.4.6 Application to Sentiment Modelling

In the present framework, the Mamba input is the concatenated vector $\tilde{z}_t = [\tilde{H}_t; \tilde{y}_t]$, where $\tilde{H}_t \in \mathbb{R}^{768}$ is the filtered FinBERT embedding and $\tilde{y}_t \in \mathbb{R}$ is the same-day standardised price. The block produces a latent state x_t and a linear readout gives the headline-level prediction:

$$\widehat{\Delta y}_t = Wx_t + b.$$

Equivalently, the full downstream predictor may be written as

$$\widehat{\Delta y}_t = f_\phi(\tilde{z}_t),$$

where f_ϕ denotes the Mamba-based state space mapping.

7.4.7 Aggregation of Multiple Headlines per Day

A trading day may contain multiple headlines, so the model produces multiple headline-level predictions for the same day. These differences represent heterogeneous effects across news items.

To obtain a single representative signal per day, all headline-level predictions corresponding to the same trading day are aggregated by averaging. Let $\{\widehat{\Delta y}_{t,1}, \widehat{\Delta y}_{t,2}, \ldots, \widehat{\Delta y}_{t,n_t}\}$ denote

the set of predicted gradients generated from n_t headlines on day t. The daily aggregated prediction is then defined as

$$\widehat{\Delta y}_t = \frac{1}{n_t} \sum_{i=1}^{n_t} \widehat{\Delta y}_{t,i}.$$

Averaging reduces headline-level noise while preserving the daily sentiment direction. The resulting single daily estimate is then aligned with the daily price series in the hybrid forecasting stage.

7.5 Modelling Assumptions

For brevity, we assume that: headlines dated at time t primarily affect the next-step gradient $\Delta y_t = y_{t+1} - y_t$; fixed FinBERT embeddings H_t preserve the financially relevant semantics needed for prediction; the relevance filter $|s_t| \geq \vartheta$ (with $\vartheta = 0.6$) removes weakly informative noise; including same-day price $\tilde{y}_t$ alongside text embedding improves conditioning of the sentiment branch; the mapping $\widehat{\Delta y}_t = f_\phi(\tilde{z}_t)$ remains sufficiently stable over the study window; averaging headline-level outputs provides a representative daily sentiment signal; and remaining unexplained variation is treated as zero-mean residual noise.

8 Combining Models

To integrate textual sentiment information with historical price dynamics, we adopt a hybrid forecasting framework that combines a daily time-series forecast with a sentiment-implied daily price signal. Both models operate at the same one-step-ahead frequency, so no blockwise interpolation or multi-day gradient aggregation is required.

8.1 Model Outputs and Notation

Let $\mathcal{T} = \{t_0, \ldots, t_N\}$ denote the trading-day grid. The time-series model produces a one-step-ahead forecast $\hat{y}_{t+1}^{(\mathrm{TS})}$, while the sentiment model predicts the daily price gradient

$$\widehat{\Delta y}_t = \hat{F}_{NLP}(\mathcal{H}_t, \tilde{y}_t), \qquad \Delta y_t = y_{t+1} - y_t.$$

Here, $\tilde{y}_t$ is the standardised same-day price feature used in the sentiment branch. This gradient is converted into a sentiment-implied price forecast via

$$\hat{y}_{t+1}^{(\mathrm{sent})} = y_t + \widehat{\Delta y}_t.$$

Thus, at each trading day t, both models provide aligned one-step-ahead forecasts for the same target y_{t+1}.

8.2 XGBoost-Based Forecast Combination

To make the combination step easy to interpret, we use a single XGBoost model to merge the two base forecasts. At each day t, it takes the time-series forecast $\hat{y}_{t+1}^{(\mathrm{TS})}$, the sentiment-implied forecast $\hat{y}_{t+1}^{(\mathrm{sent})}$, the predicted gradient $\widehat{\Delta y}_t$ and simple helper features such as their mean and difference.

The model then outputs the final one-step-ahead combined forecast:

$$\hat{y}_{t+1}^{(\mathrm{comb})} = \hat{F}_{\mathrm{XGB}}\left(\hat{y}_{t+1}^{(\mathrm{TS})}, \hat{y}_{t+1}^{(\mathrm{sent})}, \widehat{\Delta y}_t\right).$$

In practice, this lets the combiner rely more on the time-series signal during stable periods and use sentiment information more strongly when news has a larger market impact. This is important because the simplifying assumption that every headline has a direct measurable effect on price is only partially valid in the data, so the sentiment branch is used as a complementary signal rather than a standalone forecasting model [12].

9 Analysis of Combined Prediction and Conclusion

The full forecasting pipeline was executed on the dataset using an 80/20 train-test split and the following results were obtained.

	RMSE	**R^2**	**Return MSE**	**DA**
Time Series (TS)	53.9361	0.9755	0.000259	48.46%
Sentiment/News (SA)	35.5274	0.9894	0.000264	47.69%
XGBoost Combined	42.8738	0.9845	0.000263	51.54%

Table 1: Model performance comparison

The quantitative results in Table 1 show complementary strengths across the three models. The sentiment/news branch delivers the strongest price-level fit, with the lowest RMSE (35.5274) and highest R^2 (0.9894), but its return-oriented performance is weaker (Return MSE = 0.000264, DA = 47.69%). The time-series baseline achieves the lowest Return MSE (0.000259), while the XGBoost-combined model provides the best directional performance with the highest DA (51.54%) and still improves price-level accuracy over the baseline (RMSE = 42.8738, $R^2 = 0.9845$). These results suggest the sentiment/news branch is most useful as a supporting signal for fusion rather than as a standalone model.

Figure 7 shows that all three models, the baseline time-series model, the sentiment/news branch and the XGBoost-combined model, track the realised gold price path closely over the test window, with only small deviations from the observed series. The sentiment/news branch is therefore best treated as a supporting input for fusion, not as a model to deploy on its own.

Figure 7: Comparison of baseline time-series, sentiment/news and XGBoost-combined forecasts against observed gold prices on the held-out test window.

10 Conclusion

This project developed a hybrid forecasting framework that augments the baseline time-series signal with sentiment/news information and combines them using XGBoost. The results indicate that fusion improves the baseline primarily in price-level fit and directional accuracy, rather than uniformly across all return-error metrics.

Several limitations remain. First, the sentiment inputs are still a simplified representation of market news and may not fully capture the timing, intensity or persistence of real-world information shocks; relatedly, the assumption that every dated headline materially affects price is only partially supported. Second, although the combined model improves DA relative to baseline (51.54% vs. 48.46%), the gain is modest and Return MSE remains slightly above the baseline level (0.000263 vs. 0.000259), indicating that return dynamics are still not captured strongly. Third, the gains from fusion are moderate, suggesting that the interaction between sentiment and price dynamics may require more flexible modelling choices.

Future work could improve the framework by incorporating richer real-time news streams, more robust sentiment representations and adaptive fusion methods that allow the contribution of each signal to vary across market regimes. More advanced sequence models or attention-based multimodal architectures may also improve the integration of textual and numerical information.

Overall, the study shows that combining historical-price information with news-derived sentiment can improve baseline time-series forecasting for gold. The XGBoost fusion stage provides a better balance of price-level fit and directional accuracy than the standalone time-series model and offers a strong foundation for further multimodal forecasting research.

Bibliography

References

[1] G. E. P. Box, G. M. Jenkins, G. C. Reinsel and G. M. Ljung, *Time Series Analysis: Forecasting and Control*, 5th ed., Wiley, 2015. https://www.wiley.com/en-us/Time+Series+Analysis%3A+Forecasting+and+Control%2C+5th+Edition-p-9781118674918.

[2] P. J. Werbos, "Backpropagation through time: what it does and how to do it," *Proceedings of the IEEE*, 78(10):1550-1560, 1990, https://doi.org/10.1109/5.58337.

[3] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, 9(8):1735-1780, 1997, https://doi.org/10.1162/neco.1997.9.8.1735.

[4] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017, https://arxiv.org/abs/1706.03762.

[5] J. L. Ba, J. R. Kiros and G. E. Hinton, "Layer normalization," *arXiv preprint arXiv:1607.06450*, 2016, https://arxiv.org/abs/1607.06450.

[6] J. Devlin, M.-W. Chang, K. Lee and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proceedings of NAACL-HLT*, 2019, https://aclanthology.org/N19-1423/.

[7] D. Araci, "FinBERT: Financial sentiment analysis with pre-trained language models," *arXiv preprint arXiv:1908.10063*, 2019, https://arxiv.org/abs/1908.10063.

[8] A. Gu and T. Dao, "Mamba: Linear-time sequence modeling with selective state spaces," *arXiv preprint arXiv:2312.00752*, 2023, https://arxiv.org/abs/2312.00752.

[9] I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016, https://www.deeplearningbook.org/.

[10] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," *arXiv preprint arXiv:1412.6980*, 2014, https://arxiv.org/abs/1412.6980.

[11] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," *Journal of Machine Learning Research*, 15:1929-1958, 2014, https://jmlr.org/papers/v15/srivastava14a.html.

[12] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proceedings*

of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 2016, https://doi.org/10.1145/2939672.2939785.

[13] S. Bai, J. Z. Kolter and V. Koltun, "An empirical evaluation of generic convolutional and recurrent networks for sequence modeling," *arXiv preprint arXiv:1803.01271*, 2018, https://arxiv.org/abs/1803.01271.

[14] Wikipedia contributors, "Mean squared error," *Wikipedia, The Free Encyclopedia*, accessed April 6, 2026, https://en.wikipedia.org/wiki/Mean_squared_error.

[15] Wikipedia contributors, "Root mean square deviation," *Wikipedia, The Free Encyclopedia*, accessed April 6, 2026, https://en.wikipedia.org/wiki/Root_mean_square_deviation.

[16] Wikipedia contributors, "Coefficient of determination," *Wikipedia, The Free Encyclopedia*, accessed April 6, 2026, https://en.wikipedia.org/wiki/Coefficient_of_determination.

[17] Wikipedia contributors, "Mean directional accuracy," *Wikipedia, The Free Encyclopedia*, accessed April 6, 2026, https://en.wikipedia.org/wiki/Mean_directional_accuracy.